

Code review — mini-proiecte multithreading

Runde de review pe proiectul tau, cea mai noua sus. · Alex

Runda 2 — 2026-10-01 — sha 275b6cb

Poarta de compilare: trece. `javac` pe M01, M02 si M03 iese cu 0 erori. Le-am rulat pe toate trei, M03 de doua ori. Tot ce scriu mai jos e reproduc prin rulare, nu dedus.

Ce s-a inchis si ce e bine:

- ✅ **B1 din runda 1 e inchis.** `M02.java:82` tipareste acum `Vaianta A` a fost de 3.8ori mai rapid, cu `durataB/durataA` si rotunjire la o zecimala. Exact cum trebuia.
- 🟢 **Prima data cand folosesti `implements Runnable`.** La M01 si M02 ai mers pe `extends Thread`. La M03 ai separat `munca` (`Comanda`, `Jurnal`) de `firul care o duce` (`new Thread(comanda, "Comanda-7")`). Asta e fix elementul `B1-runnable` din teorie, si l-ai luat singur.
- 🟢 Ai dat nume firelor si le tiparesti. Output-ul se citeste.
- 🟢 In `catch (InterruptedException)` pui inapoi steagul cu `Thread.currentThread().interrupt()`. Putini fac asta.

M03 are insa trei probleme, si fiecare rupe cate un criteriu de acceptare. Le iau pe rand.

Critice

B1 🟢 Jurnalul incrementeaza contorul pe care avea voie doar sa-l citeasca

`src/M03-jurnal-de-fundal/M03.java:61` (si lipsa din `:38-47`)

CUM E ACUM: rulat de doua ori, ambele rulari identice la final:

```
Comanda : 8procesata de: Comanda-8
SUMAR: 1 comenzi procesate.
```

Deasupra sunt 20 de linii `Comanda : N procesata`. Programul numara una. Iar singura linie de jurnal din toata rularea scrie `Jurnal procesate : 0`, desi pana in acel moment se terminasera deja vreo zece comenzi.

DE CE: `getAndIncrement()` face doua lucruri intr-un singur apel — intoarce valoarea curenta **si** o creste cu 1. In `Jurnal.run()` tu ai nevoie doar de prima jumatate; a doua se executa oricum. Deci cine creste contorul e **cititorul**, nu lucratorul: `Comanda.run()` isi tipareste mesajul la linia 43 si pleaca fara sa atinga `procesate`.

Rezultatul: `procesate` nu numara comenzi, numara **cate ture a facut jurnalul**. Jurnalul a apucat o singura tura (doarme 100 ms, iar totul s-a terminat in ~200 ms), deci la final `procesate` e 1 — de unde `SUMAR: 1`. Iar `0`-ul din linia de jurnal vine din aceeaasi linie: `getAndIncrement` intoarce valoarea **dinainte** de crestere.

Doua numere gresite, din aceeaasi instructiune. Semnalul: daca un contor e citit intr-un loc si crescut in acelasi loc, nimeni nu-l mai creste acolo unde se intampla munca.

FIX: cresterea apartine lucratorului, citirea cititorului.

acum	corect
<code>Comanda.run()</code> nu atinge <code>procesate</code>	<code>procesate.incrementAndGet();</code> dupa <code>sleep</code>
<code>... + procesate.getAndIncrement()</code>	<code>... + procesate.get()</code>

B2 ● Pornesti cele 20 de comenzi deodata, desi cerinta zice "una dupa alta"

src/M03-jurnal-de-fundal/M03.java:81-90

CUM E ACUM: cronometrat pe cele doua rulari: **205 ms** si **219 ms**. Criteriul de acceptare cere ca programul sa se termine *"in aproximativ timpul celor 20 de comenzi"* — adica $20 \times \sim 100 \text{ ms} \approx \mathbf{2000 \text{ ms}}$. Si cere *"linii de jurnal intercalate cu comenzile"*, tu ai **o singura** linie de jurnal la 20 de comenzi.

DE CE: pornite in paralel, cele 20 de fire nu dureaza cat **suma** duratelor, ci cat **cea mai lenta** dintre ele — maximum 150 ms. Jurnalul bate din 100 in 100 ms, deci apuca exact o bataie si gata. Intercalarea nu e un detaliu de afisare: e singura dovada ca jurnalul raporteaza **in timp ce** se lucreaza, nu dupa. Cand munca se termina inainte ca jurnalul sa apuce a doua tura, n-ai cu ce sa intercalezi.

Ai aplicat aici solutia de la M01. La M01 paralelismul **era** cerinta — 5 descarcari care trebuie sa se suprapuna. La M03 munca e secventiala intentionat, si singurul fir in plus e jurnalul. Cele doua proiecte seamana la suprafata (`List<Thread>`, `start()`, `join()`) si cer lucruri opuse.

FIX: comenzile raman pe `main`, una dupa alta.

acum	corect
<code>for (...) { thread.start(); } + join()</code>	<code>for (int i = 1; i <= 20; i++) {</code>
20 de obiecte <code>Thread</code>	<code>Thread.sleep(50 + r.nextInt(101));</code>
total 205 ms	<code>procesate.incrementAndGet();</code>
1 linie de jurnal	<code>System.out.println("comanda " + i); }</code>

Singurul `new Thread` ramas in program e jurnalul.

B3 ● Jurnalul nu e daemon, deci programul il asteapta

src/M03-jurnal-de-fundal/M03.java:75-76 si :96-98

CUM E ACUM: programul chiar se termina, dar nu de la sine: se termina pentru ca il **opresti tu** (`jurnalThread.interrupt()`) si apoi **il astepti** (`jurnalThread.join()`). Cerinta spune insa doua lucruri: jurnalul *"scrie la nesfarsit, cat timp exista aplicatie"* si programul *"se INCHIDE IMEDIAT. Nu are voie sa mai astepte nimic."*

Am verificat pe fisierul tau, fara sa-l modific in repo. Am facut o copie si am sters cele doua linii, `interrupt()` si `join()`:

```
Jurnal procesate : 56
INCA RULEAZA dupa 6 secunde -> atarna
```

Apoi, pe aceeasi copie fara `interrupt / join`, am adaugat **o singura linie** inainte de `start()`:

```
jurnalThread.setDaemon(true);
--> SUMAR afisat, program terminat singur in 201 ms
```

DE CE: JVM-ul nu se inchide cand se termina `main`. Se inchide cand se termina **ultimul fir non-daemon**. Orice fir pe care il creezi e non-daemon implicit, deci jurnalul tau e un motiv ca masina virtuala sa ramana in viata — la nesfarsit, fiindca bucla lui n-are iesire. De-aia, fara `interrupt()`, programul atarna.

`setDaemon(true)` muta firul pe cealalta lista: cea pe care JVM-ul **n-o numara** la inchidere. Cand ultimul fir normal se termina, firele daemon sunt taiate pe loc, oriunde ar fi. De-aia steagul trebuie pus **inainte** de `start()` — dupa pornire firul e deja in scris pe lista cealalta si `setDaemon` arunca `IllegalThreadStateException`.

`interrupt()` + `join()` e solutia corecta la **alta** intrebare: "cum opresc un fir pe care vreau sa-l astept". Tu raspunzi la "cum il opresc", cand intrebarea proiectului era "cum fac sa nu trebuiasca sa-l astept". Demo-ul `teorie/A5-daemon` are si el `interrupt()` + `join()` la final, dar acolo e doar ca sa nu curga output la infinit in timpul lectiei — linia care conteaza e `tick.setDaemon(true)`, cu tot cu comentariul de deasupra.

FIX: `jurnalThread.setDaemon(true);` inainte de `start()`; sterge `interrupt()` si `join()`.

Ramase deschise din runda 1 — niciuna atinsa

Pe toate patru ai trecut pe langa ele cand ai reparat B1.

#	Unde	Ce
M1 	<code>M02.java:72-75</code>	<code>run()</code> pe fire deja consumate de <code>start()</code> + <code>join()</code>
M2 	<code>M01.java:82</code>	<code>System.out.println("test")</code> ramas in mijlocul raportului
C1 	<code>M01.java:92-93</code>	citesti <code>pagina.durata</code> direct, desi ai getteri la 91
C2 	<code>M02.java:63,79,83</code>	Vaianta <code>x3</code> , Medtoda la 41, lipsa spatiu: <code>3.8ori</code>

Si asta e lucrul de retinut al rundeii. Fixul lui B1 l-ai scris pe `M02.java:82`. Pe linia **83**, imediat sub cursor, sunt doua dintre constatările C2: `Vaianta` si `"ori mai rapid "` fara spatiu. Le-ai avut pe ecran in timp ce reparai si au ramas acolo.

E a cincea oara cand apare tiparul: repara **linia semnalata**, nu **regula**. La recap: ai adus inapoi `close()` dar nu l-ai chemat; ai promovat `inTransit` la camp dar nu l-ai incrementat. In runda 1: ai tiparit propozitia dar nu si N-ul calculat. Acum: ai reparat linia 82 si ai lasat 83.

Propunerea mea, concreta: cand primesti reviewul, nu lua constatările pe rand. Deschide fiecare fisier mentionat **o singura data** si rezolva tot ce e in el, de sus in jos. Patru constatari din patru erau in doua fisiere.

Intrebari de verificare

- Programul tau a tiparit 20 de linii `Comanda : N procesata` si apoi `SUMAR: 1 comenzi procesate`. De unde vine cifra 1 — ce anume a numarat contorul, daca nu comenzi?
- Daca stergi `jurnalThread.interrupt()` si `jurnalThread.join()` din codul tau de acum, programul nu se mai opreste niciodata. Daca adaugi in schimb `setDaemon(true)`, se opreste in 200 ms. Ce face `main` diferit in cele doua cazuri? (Indiciu: nimic. Se schimba cine decide inchiderea.)
- M01 cere paralelism, M03 cere munca secventiala cu un singur fir in fundal. Cum ai fi putut sa-ti dai seama din **criteriul de acceptare**, inainte de a scrie o linie, ca M03 nu e M01?

Urmatorul pas: repara M03 (cele trei  sunt in acelasi fisier), inchide cele patru ramase din runda 1, apoi M04 si M05.

Runda 1 — 2026-09-29 — sha d845a33

M01 si M02, livrate la mai putin de o ora dupa primirea lotului. Poarta de compilare: trece.

-  **B1 inchis** (runda 2) — `M02.java:82`, verdictul tiparea fractia rasturnata: `0.26` in loc de `3.8`.

- **M1 NEATINS** — M02.java:72–75 , `run()` pe fire deja pornite si terminate. Merge doar fiindca `run()` e metoda obisnuita; `start()` a doua oara ar da `IllegalThreadStateException` .
- **M2 NEATINS** — M01.java:82 , `println("test")` in mijlocul raportului.
- **C1 NEATINS** — M01.java:92–93 , getteri scrisi si apoi ocoliti prin acces direct la camp.
- **C2 NEATINS** — `Vaianta` x3, `Medtoda` , lipsa spatiu la `3.8ori` .

Bine in runda 1: la M01 `join()` in bucla **separata** de `start()` (:77 si :80) — miezul proiectului; verdict in `paralel` , 1061 ms fata de o suma de 3832 ms. La M02, diferenta de **un singur cuvant** intre variante (`start()` la 50, `run()` la 72), confirmata de numele firelor.